

**Автономная некоммерческая организация высшего образования
«Университет Иннополис»**

**ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА
(БАКАЛАВРСКАЯ РАБОТА)
по направлению подготовки
09.03.01 - «Информатика и вычислительная техника»**

**GRADUATION THESIS
(BACHELOR'S GRADUATION THESIS)**

**Field of Study
09.03.01 – «Computer Science»**

**Направленность (профиль) образовательной программы
«Информатика и вычислительная техника»
Area of Specialization / Academic Program Title:
«Computer Science»**

**Тема /
Topic**

**Visual Studio Code Extension for Type-Based Visualization for
the Stella Programming Language / Расширение Visual Studio
Code для визуализации информации на основе типов для
языка программирования Stella**

**Работу выполнил /
Thesis is executed by**

**Фадеенков Антон
Олегович / Fadeenkov Anton
Olegovich**

подпись / signature

**Руководитель
выпускной
квалификационной
работы /
Supervisor of
Graduation Thesis**

**Кудасов Николай
Дмитриевич /
Kudasov Nikolay Dmitrievich**

подпись / signature

Введение, цель и задачи

Выпускная квалификационная работа посвящена разработке расширения Visual Studio Code для визуализации результатов синтаксического и типового анализа программ на языке Stella. Stella является учебным статически типизированным языком, который используется при изучении систем типов и принципов построения компиляторов. Для таких курсов важно видеть не только итоговую ошибку или выведенный тип, но и промежуточные структуры анализа: абстрактное синтаксическое дерево, контекст типизации, дерево вывода типов и зависимости между выражениями.

Проблема состоит в том, что стандартные средства редактора обычно показывают только конечный результат работы языкового сервера: подсказку, подчеркивание ошибки или краткое диагностическое сообщение. В результате студенту приходится самостоятельно восстанавливать, какие подвыражения участвовали в проверке и почему было получено суждение $\Gamma \vdash e : T$, где Γ обозначает контекст типизации, e — выражение, а T — его тип.

Цель работы — спроектировать и реализовать расширение Visual Studio Code, которое представляет результаты анализа Stella в виде интерактивных представлений, связанных с исходным кодом. Для достижения цели были изучены подходы к образовательной визуализации, определены необходимые представления, спроектирован обмен данными между клиентом и языковым сервером, реализованы пользовательские запросы Language Server Protocol и проведена функциональная проверка прототипа.

Обзор литературы

Визуализация программ применяется в обучении информатике для объяснения структур и процессов, которые трудно наблюдать напрямую. Работы по software visualization и algorithm visualization показывают, что интерактив-

ные представления могут поддерживать понимание абстрактных процессов, однако их результаты нельзя автоматически переносить на конкретный инструмент без отдельной проверки. Поэтому исследования Price, Hundhausen, Naps, Sorva и Kogan используются в работе как основание для проектирования, а не как прямое доказательство эффективности расширения.

Близкими по тематике являются Jeliot и visAST. Jeliot визуализирует выполнение программ, а visAST связывает исходный текст с абстрактным синтаксическим деревом. Отличие данной работы состоит в том, что разработанное расширение охватывает не только синтаксис, но и информацию, связанную с типами: контекст типизации, дерево вывода, трассу ошибки и граф зависимостей. Теоретическую основу составляют работы по статическим системам типов, суждениям типизации и анализу программ.

Краткое описание основного содержания работы

В работе разработан прототип расширения Visual Studio Code для языка Stella. Клиентская часть реализует пользовательские представления редактора, а серверная часть подготавливает данные анализа и передает их через расширенные запросы Language Server Protocol. Для обмена используется JSON-RPC, что согласуется с архитектурой LSP и позволяет отделить вычисление результатов анализа от их отображения в интерфейсе.

Основными результатами являются пять представлений. AST View показывает структуру программы в виде дерева. Scope View отображает область видимости и контекст типизации. Type Derivation View показывает последовательность применения правил вывода типов. Type Error Trace View помогает проследить путь к ошибке типизации. Type Dependency Graph показывает зависимости между выражениями и типовыми суждениями; такое представление связано с идеями program dependence graph и program slicing, но адаптировано к учебному языку Stella.

Отдельное внимание уделено согласованию технологического стека с принципами проектирования. TypeScript выбран как основной язык реализации из-за строгой типизации и совместимости с экосистемой Visual Studio Code. Langium используется для реализации языковой инфраструктуры, а API Visual Studio Code — для построения Tree View и Webview-представлений. Такой стек позволяет сохранить связь между анализом, редактором и пользовательским интерфейсом без отдельного внешнего приложения.

Проверка результата выполнена функционально: были рассмотрены типовые программы Stella, корректность ответов языкового сервера и отображение результатов в интерфейсе редактора. Полноценное пользовательское исследование не проводилось, поэтому выводы об удобстве и влиянии на понимание ограничены. Это ограничение явно учитывается как угроза корректности результатов.

Заключение

В результате работы было создано расширение Visual Studio Code, которое визуализирует промежуточные результаты анализа программ на языке Stella. Расширение показывает не только итоговый результат проверки, но и структуры, которые участвуют в синтаксическом и типовом анализе. Это делает прототип полезным как вспомогательный инструмент для изучения реализации систем типов.

Основной вклад работы состоит в проектировании набора представлений для Stella, реализации обмена данными между расширением и языковым сервером, создании пользовательского интерфейса в Visual Studio Code и функциональной проверке прототипа. Дальнейшее развитие работы может включать полноценное пользовательское исследование, оценку удобства по SUS, улучшение визуализации графов и расширение поддержки новых кон-

струкций языка Stella.

Список использованных источников

- [1] Microsoft. Language Server Protocol Specification 3.17. URL: <https://microsoft.github.io/language-server-protocol/specifications/lsp/3.17/specification/>.
- [2] JSON-RPC Working Group. JSON-RPC 2.0 Specification. URL: <https://www.jsonrpc.org/specification>.
- [3] Microsoft. Visual Studio Code Extension API. URL: <https://code.visualstudio.com/api>.
- [4] Microsoft. Visual Studio Code Tree View API. URL: <https://code.visualstudio.com/api/extension-guides/tree-view>.
- [5] Microsoft. Visual Studio Code Webview API. URL: <https://code.visualstudio.com/api/extension-guides/webview>.
- [6] TypeFox. Langium Documentation. URL: <https://langium.org/docs/>.
- [7] Kudasov N. Stella Language Documentation. URL: <https://fizruk.github.io/stella/site/core/introduction/>.
- [8] Abounegm A., Kudasov N., Stepanov A. Teaching Type Systems Implementation with STELLA. 2024. URL: <https://arxiv.org/abs/2407.08089>.
- [9] Pierce B. C. Types and Programming Languages. MIT Press, 2002.
- [10] Harper R. Practical Foundations for Programming Languages. Cambridge University Press, 2016.
- [11] Nielson F., Nielson H. R., Hankin C. Principles of Program Analysis. Springer, 1999.
- [12] Price B. A., Baecker R. M., Small I. S. A Principled Taxonomy of Software Visualization // Journal of Visual Languages and Computing. 1993.

- [13] Hundhausen C. D., Douglas S. A., Stasko J. T. A Meta-Study of Algorithm Visualization Effectiveness // Journal of Visual Languages and Computing. 2002.
- [14] Naps T. L. et al. Exploring the Role of Visualization and Engagement in Computer Science Education // ITiCSE Working Group Reports. 2002.
- [15] Sorva J., Karavirta V., Malmi L. A Review of Generic Program Visualization Systems // ACM Transactions on Computing Education. 2013.
- [16] Moreno A., Myller N., Sutinen E., Ben-Ari M. Visualizing Programs with Jeliot 3 // AVI. 2004.
- [17] Aalvik R. visAST: Generic AST Visualiser for Software Language Education: master's thesis. University of Bergen, 2019.
- [18] Ferrante J., Ottenstein K. J., Warren J. D. The Program Dependence Graph and Its Use in Optimization // ACM TOPLAS. 1987.
- [19] Brooke J. SUS: A Quick and Dirty Usability Scale // Usability Evaluation in Industry. Taylor & Francis, 1996.
- [20] Bangor A., Kortum P., Miller J. Determining What Individual SUS Scores Mean // Journal of Usability Studies. 2009.
- [21] Wohlin C. et al. Experimentation in Software Engineering. Springer, 2012.
- [22] Runeson P., Hoest M. Guidelines for Conducting and Reporting Case Study Research in Software Engineering // Empirical Software Engineering. 2009.